Data Structures & Algorithms — Study Notes

Unit 3: Linear and Non-Linear Structures

1. Arrays

An array is a fixed-size, contiguous block of memory that stores elements of the same type. Because elements sit next to each other in memory, any element can be accessed directly using its index, making lookups very fast — O(1) time complexity regardless of array size.

The tradeoff is flexibility. Inserting or deleting an element in the middle of an array requires shifting every subsequent element, which takes O(n) time in the worst case. Arrays also have a fixed size once allocated, which is why dynamic arrays (like Python lists or Java ArrayLists) exist — they allocate extra capacity behind the scenes and resize when full.

1.1 Two Pointers

The two-pointer technique uses two index variables that move through an array, often from opposite ends or at different speeds, to solve problems in a single pass instead of using nested loops. It's commonly used for problems like finding a pair that sums to a target in a sorted array, or reversing an array in place.

1.2 Sliding Window

A sliding window maintains a subrange of an array (a 'window') that expands or contracts as it moves across the data, avoiding recomputation from scratch at each step. This technique builds directly on array indexing and is especially useful for problems involving contiguous subarrays, such as finding the longest substring without repeating characters.

2. Hash Maps and Sets

A hash map stores key-value pairs and uses a hash function to convert a key into an index within an internal array, allowing average-case O(1) insertion, deletion, and lookup. This makes hash maps dramatically faster than arrays for lookups when the data isn't naturally ordered by index.

A hash set is a hash map that only stores keys, no values — useful for quickly checking whether an element has already been seen. Both structures depend on the same underlying hashing concept, and understanding basic array indexing is a prerequisite to understanding how a hash table's internal buckets work.

3. Linked Lists

Unlike arrays, a linked list does not store its elements contiguously in memory. Instead, each element (called a node) stores its own value plus a reference (or 'pointer') to the next node in the sequence. This makes insertion and deletion at any position O(1) once you have a reference to the right spot, since no shifting is required — the tradeoff is that finding a specific element requires walking the list from the start, which is O(n).

A doubly linked list extends this idea by giving each node a reference to both the next and previous node, allowing traversal in both directions at the cost of extra memory per node.

4. Stacks and Queues

A stack is a Last-In-First-Out (LIFO) structure: the most recently added element is the first one removed. Stacks are commonly implemented using either an array or a linked list, and are used for problems involving nested structures, such as matching parentheses or implementing undo functionality.

A queue is a First-In-First-Out (FIFO) structure: elements are removed in the order they were added. Queues are essential for breadth-first traversal algorithms, which are covered later in this unit under graph and tree traversal.

5. Trees

A tree is a hierarchical, non-linear data structure made of nodes connected by edges, where each node has exactly one parent (except the root, which has none) and zero or more children. Trees generalize the idea of a linked list — instead of each node pointing to a single 'next' node, a tree node can branch into multiple children, which is why understanding linked lists first makes trees far more intuitive.

Key terminology: the root is the topmost node, a leaf is a node with no children, and the height of a tree is the number of edges on the longest path from root to leaf.

5.1 Binary Search Trees (BST)

A binary search tree is a tree where each node has at most two children, and — critically — every node's left subtree contains only values smaller than the node, and its right subtree contains only values larger. This ordering property is what allows searching, insertion, and deletion to run in O(log n) time on a balanced tree, since each comparison eliminates roughly half of the remaining nodes, similar in spirit to binary search on a sorted array.

If a BST becomes unbalanced (for example, by inserting already-sorted data), it degrades toward a linked list, and operations slow to O(n). Self-balancing variants like AVL trees and Red-Black trees exist specifically to prevent this, though the rebalancing logic is beyond the scope of this unit.

5.2 Binary Search (on sorted arrays)

Binary search is an algorithm — not a data structure — for finding a target value in a sorted array by repeatedly comparing the target to the middle element and discarding the half of the array that cannot contain it. It runs in O(log n) time and is the array-based cousin of BST search: both rely on the same halving principle, just applied to different underlying structures.

5.3 Tree Traversal: BFS and DFS

Breadth-First Search (BFS) visits a tree level by level, exploring all nodes at the current depth before moving to the next. It is implemented using a queue, which is why understanding queues is a prerequisite — each node is enqueued when discovered and dequeued when processed.

Depth-First Search (DFS) explores as far as possible down one branch before backtracking. It is commonly implemented using a stack (or recursion, which uses the call stack implicitly), which is why understanding stacks first makes DFS easier to follow. DFS has three common orderings on binary trees: pre-order, in-order, and post-order, depending on when the current node is processed relative to its children.

6. Heaps

A heap is a specialized tree-based structure that satisfies the heap property: in a min-heap, every parent node is smaller than or equal to its children (the opposite for a max-heap). This guarantees the smallest (or largest) element is always at the root, making heaps ideal for implementing priority queues. Because a heap is a form of binary tree, understanding basic tree structure and terminology is necessary before heaps will make sense.

Insertion and removal from a heap run in O(log n) time, since maintaining the heap property after a change only requires moving an element up or down the height of the tree, not searching the whole structure.

7. Greedy Algorithms

A greedy algorithm builds a solution step by step, always choosing the option that looks best at the current moment, without reconsidering earlier choices. Greedy approaches are fast and simple, but only produce a correct, optimal solution for problems that have what's called the 'greedy choice property' — not every problem has this, which is a common source of bugs when greedy logic is applied somewhere it doesn't actually hold.

8. Basic Dynamic Programming

Dynamic programming (DP) solves problems by breaking them into overlapping subproblems, solving each subproblem once, and storing the result (memoization) so it doesn't need to be recomputed. This is fundamentally different from greedy algorithms — DP explores multiple possibilities and keeps the best one, rather than committing to a single choice at each step. A basic grasp of recursion is a prerequisite, since most DP solutions start as a recursive formulation before being optimized with memoization or converted to an iterative table-filling approach.

A classic introductory example is the Fibonacci sequence: computing it naively with plain recursion recalculates the same values repeatedly, resulting in exponential time complexity. Storing each computed value the first time it's calculated reduces this to linear time.